



机器学习与 Python 实践

第四次作业

基于学习行为数据的学生成绩等级分类 模型

姓名：

专业： 强基班

学号：

教师：

学院： 信息科学与工程学院

2026 年 6 月 5 日

目录

1	前言及问题概述	2
2	相关基础知识及方法	2
2.1	相关原理	2
2.2	数据描述	3
2.3	算法实现	5
3	实验结果与可扩展内容	6
3.1	实验结果	6
3.2	参数调优及可扩展内容	10
4	讨论与结论	11
4.1	结果讨论	11
4.2	结论总结	11
5	心得体会与思考	11
5.1	个人心得	11
5.2	启发与思考	12
6	附录	12
	算法代码及其注释	12
	关键运行结果汇总	28

1 前言及问题概述

随着在线学习平台和移动端学习管理系统的发展，学生在学习过程中的行为数据能够被较为完整地记录下来。例如，上课举手次数、访问课程资源次数、查看课程公告次数、参与讨论次数、家长是否参与问卷、缺勤情况等信息，都可以从不同角度反映学生的学习投入程度和外部支持情况。若能利用这些数据对学生成绩等级进行预测，可以帮助教师更早发现潜在风险学生，并为教学干预、家校沟通和学习资源推送提供参考。

本次实验使用题目给出的 `kalboard360_xAPI_Edu_Data.csv` 数据集。该数据来自手机 APP “Kalboard 360” 的学习管理系统，平台旨在利用云端技术提升 K-12 教育中的学习管理水平。数据集共包含 480 条学生记录和 16 个输入特征，目标变量为 `Class`，表示学生成绩等级，取值包括 L、M 和 H，分别代表低、中、高三个等级。

题目要求是根据学习者在学习活动中的特征预测学生成绩等级。因此，本实验属于监督学习中的多分类问题。实验过程包括数据读取、数据预览、探索性分析、特征工程、类别编码、模型训练、模型评价和可视化展示。为了使结果更可靠，本文同时比较多数类基线、逻辑回归、支持向量机、随机森林和梯度提升等模型，并使用准确率、宏平均 F1 值、平衡准确率和混淆矩阵综合评价模型效果。

2 相关基础知识及方法

2.1 相关原理

分类任务的目标是根据输入特征向量 x 判断样本所属类别 y 。在本实验中， x 表示一名学生的学习行为、人口统计和教育背景特征， $y \in \{L, M, H\}$ 表示学生成绩等级。分类模型可以表示为

$$\hat{y} = f(x), \quad (1)$$

其中 $\hat{y}$ 是模型预测的类别。模型训练的核心目标是从已有标注样本中学习类别边界，使模型在未见过的测试样本上仍能保持较好的判断能力。

逻辑回归是常用的线性分类模型。对于多分类任务，可以将二分类逻辑回归扩展为多项逻辑回归，模型输出每个类别的概率，并选择概率最高的类别作为预测结果。逻辑回归训练速度快、解释性较好，但其决策边界主要是线性的，对复杂非线性关系表达能力有限。

支持向量机通过寻找能够最大化类别间隔的分类超平面来完成分类。当使用 RBF 核函数时，支持向量机可以在高维特征空间中形成非线性分类边界，适合处理中小规模、类别边界较复杂的数据。随机森林则由多棵决策树集成而成，通过样本和特征的随机抽样降低过拟合风险；梯度提升模型同样基于决策树，但采用逐步拟合残差的方式不断提高模型表现。

由于本数据集的目标类别不是完全均衡，仅使用准确率可能会掩盖少数类别的预测问题。因此，本文重点使用宏平均 F1 值和平衡准确率。对于单个类别，精确率、召回率和 F1 值分别定义为

$$Precision = \frac{TP}{TP + FP}, \quad (2)$$

$$Recall = \frac{TP}{TP + FN}, \quad (3)$$

$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}. \quad (4)$$

宏平均 F1 是对所有类别 F1 值直接取平均，因此每个类别的权重相同，更能反映模型对三类学生的整体识别能力。

2.2 数据描述

数据集文件为 `kalboard360_xAPI_Edu_Data.csv`，读取后共有 480 行、17 列，其中 16 列为输入特征，1 列为目标变量。数据中没有缺失值，因此不需要进行额外缺失值填补。主要字段含义见表 1。

表 1: Kalboard 360 学生成绩等级数据集字段说明

字段	类型	含义
gender	类别特征	学生性别
NationalITy	类别特征	学生国籍
PlaceofBirth	类别特征	学生出生地
StageID	类别特征	教育阶段，如小学低年级、中学等
GradeID	类别特征	年级编号
SectionID	类别特征	所属班级
Topic	类别特征	学习科目
Semester	类别特征	学期
Relation	类别特征	主要监护人关系
raisedhands	数值特征	上课举手次数
VisITedResources	数值特征	访问课程资源次数
AnnouncementsView	数值特征	查看公告次数
Discussion	数值特征	参与讨论次数
ParentAnsweringSurvey	类别特征	家长是否回答问卷
ParentschoolSatisfaction	类别特征	家长对学校满意度
StudentAbsenceDays	类别特征	学生缺勤天数是否超过 7 天
Class	目标变量	学生成绩等级，分为 L、M、H

目标变量 `Class` 的分布如图 1 所示，其中低等级 `L` 有 127 条，占 26.46%；中等级 `M` 有 211 条，占 43.96%；高等级 `H` 有 142 条，占 29.58%。中等级样本最多，但三类样本数量差距不算极端，因此可以使用分层划分训练集和测试集，并采用宏平均指标评价模型。

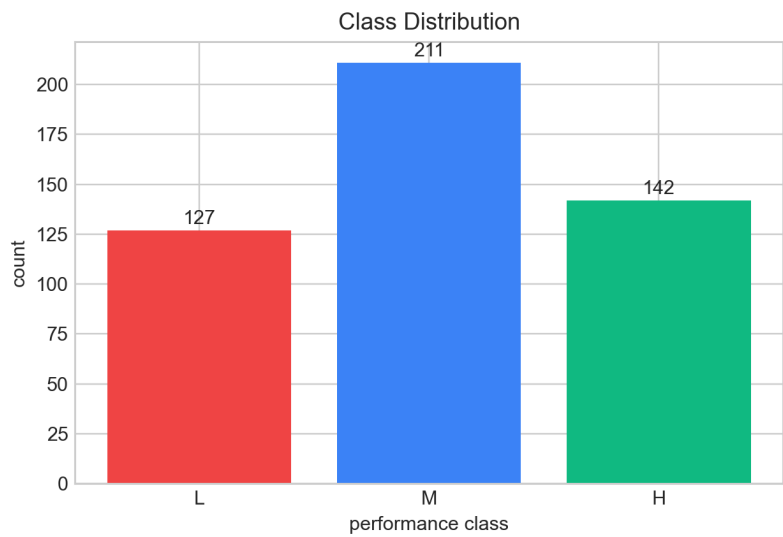


图 1: 学生成绩等级类别分布

学习行为变量在不同成绩等级下的均值见表 2。可以看出，高等级学生在举手、访问资源、查看公告和参与讨论四类行为上均明显高于低等级学生。例如，`H` 类学生平均访问课程资源次数为 78.75，而 `L` 类学生仅为 18.32。这说明学习活跃度与成绩等级之间存在较强联系。

表 2: 不同成绩等级下主要学习行为均值

成绩等级	举手次数	访问资源	查看公告	参与讨论
L	16.89	18.32	15.57	30.83
M	48.94	60.64	40.96	43.79
H	70.29	78.75	53.38	53.66

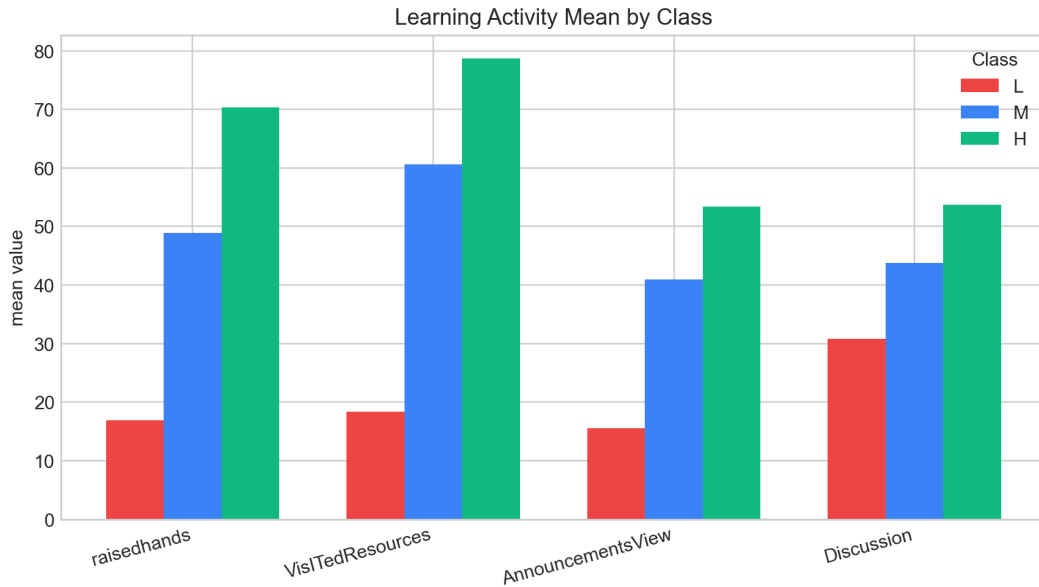


图 2: 不同成绩等级下学习行为均值对比

2.3 算法实现

实验程序使用 `pandas` 读取 CSV 数据, 使用 `scikit-learn` 完成特征预处理、模型训练、交叉验证和模型评价。主要流程如下:

1. 读取 `kalboard360_xAPI_Edu_Data.csv`, 确认数据规模、字段类型、缺失值和目标类别分布。
2. 保存前 10 行数据、字段类型、缺失值统计、描述性统计、类别分布和不同成绩等级下的学习行为统计。
3. 绘制目标类别分布图、学习行为均值图、学习行为箱线图和数值特征相关性热力图。
4. 构造补充特征, 包括 `activity_total`、`activity_mean`、`resource_per_raise`、`absence_above_7`、`parent_answer_yes`、`absence_parent_risk` 等。
5. 将数据按 80%/20% 分层随机划分为训练集和测试集, 其中训练集 384 条, 测试集 96 条。
6. 对类别特征使用 One-Hot 编码, 对数值特征使用标准化处理。
7. 分别训练 `Majority_Baseline`、`LogisticRegression`、`SVM_RBF`、`RandomForest` 和 `GradientBoosting` 五个模型。
8. 在测试集上计算 `Accuracy`、`Macro Precision`、`Macro Recall`、`Macro-F1` 和 `Balanced Accuracy`, 同时使用 5 折分层交叉验证估计模型稳定性。

9. 对最佳模型保存混淆矩阵、分类报告和特征重要性图。

完整 Python 程序见附录。

3 实验结果与可扩展内容

3.1 实验结果

图 3 展示了四个学习行为变量在不同成绩等级下的箱线图。整体来看，L 类学生的举手次数和资源访问次数集中在较低区间，H 类学生明显更高，M 类学生位于二者之间。这说明学习行为数据可以为成绩等级分类提供有效信息。

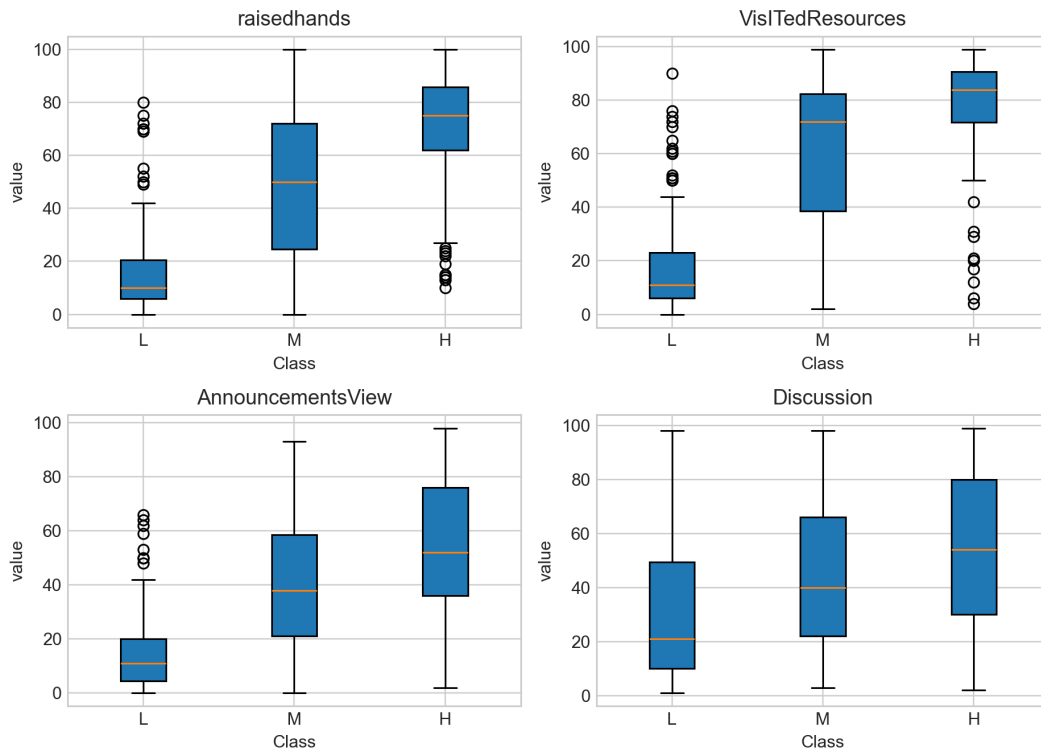


图 3: 主要学习行为在不同成绩等级下的箱线图

数值特征相关性热力图见图 4。将目标等级映射为 L=0、M=1、H=2 后，可以看到 `raisedhands`、`VisITedResources`、`AnnouncementsView` 和 `Discussion` 与成绩等级整体呈正相关。其中，访问课程资源和举手次数的区分作用最明显。

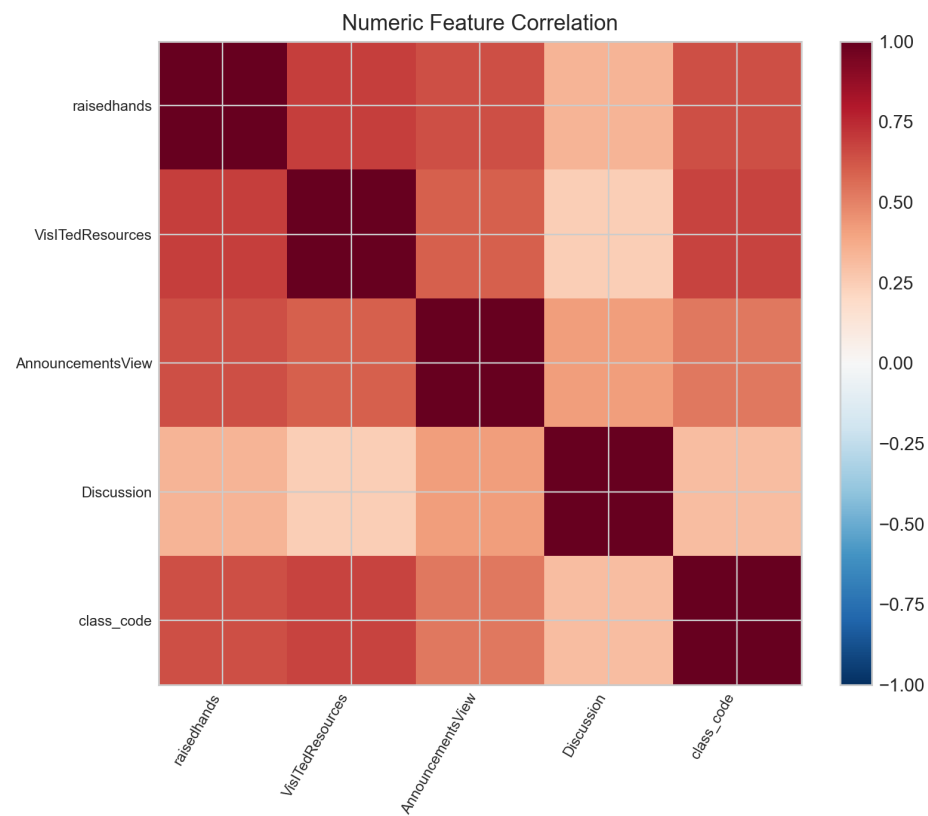


图 4: 数值特征相关性热力图

不同模型在测试集上的评价结果见表 3。多数类基线模型只预测样本最多的 M 类，准确率为 0.4375，Macro-F1 仅为 0.2029，说明只依赖类别先验无法完成有效分类。加入学习行为、背景信息和家校互动特征后，所有机器学习模型均明显优于基线。

表 3: 模型测试集评价结果

模型	Acc.	Macro-P	Macro-R	Macro-F1	Bal. Acc.
SVM_RBF	0.8021	0.8066	0.8117	0.8090	0.8117
RandomForest	0.7604	0.7665	0.7764	0.7707	0.7764
GradientBoosting	0.7396	0.7455	0.7588	0.7513	0.7588
LogisticRegression	0.7292	0.7298	0.7580	0.7385	0.7580
Majority_Baseline	0.4375	0.1458	0.3333	0.2029	0.3333

从表中可以看到，SVM_RBF 表现最好，测试集准确率为 0.8021，Macro-F1 为 0.8090，平衡准确率为 0.8117。随机森林次之，Macro-F1 为 0.7707。支持向量机优于树模型，说明该小样本数据中不同等级之间可能存在较复杂但相对平滑的非线性分类边界，RBF 核函数能够较好地捕捉这种边界。

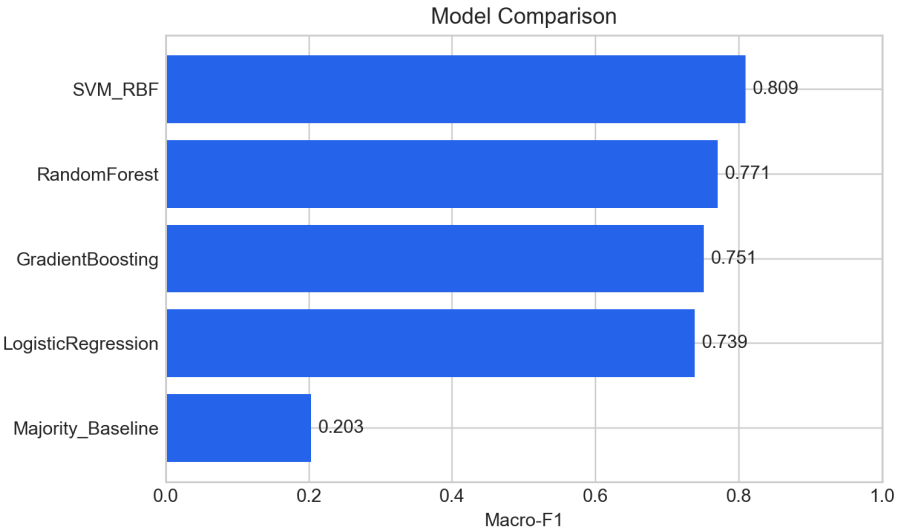


图 5: 不同模型 Macro-F1 对比

表 4 给出了 5 折分层交叉验证结果。交叉验证中 SVM_RBF 的平均准确率为 0.7833, 平均 Macro-F1 为 0.7902, 平均平衡准确率为 0.7917, 与测试集结果较接近, 说明模型表现具有一定稳定性。随机森林的交叉验证 Macro-F1 为 0.7866, 与 SVM 非常接近, 也说明树模型同样是本任务中较可靠的选择。

表 4: 模型 5 折分层交叉验证结果

模型	CV Accuracy	CV Macro-F1	CV Balanced Acc.
SVM_RBF	0.7833	0.7902	0.7917
RandomForest	0.7792	0.7866	0.7876
GradientBoosting	0.7417	0.7486	0.7465
LogisticRegression	0.7250	0.7328	0.7444
Majority_Baseline	0.4396	0.2036	0.3333

最佳模型 SVM_RBF 的混淆矩阵见图 6。测试集共有 96 条样本, 其中 L 类 25 条、M 类 42 条、H 类 29 条。模型正确识别了 22 条 L 类、32 条 M 类和 23 条 H 类样本。主要错误发生在 M 类与相邻等级之间, 这符合中间等级边界相对模糊的特点。

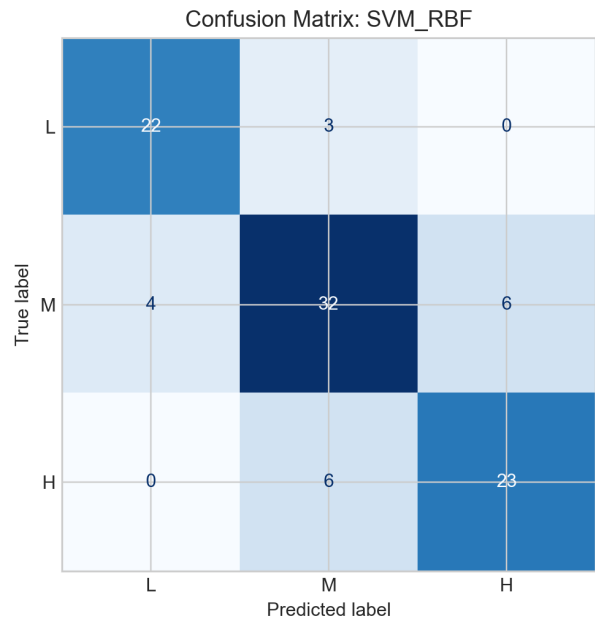


图 6: SVM_RBF 测试集混淆矩阵

从分类报告看，L 类的 F1 值为 0.8627，M 类的 F1 值为 0.7711，H 类的 F1 值为 0.7931。低等级学生识别效果最好，这对于实际教学预警具有一定意义，因为低成绩风险学生往往是最需要优先识别和干预的对象。

图 7 给出了最佳模型的前 20 个特征重要性。由于 SVM 本身不直接提供树模型式的特征重要性，实验采用基于 Macro-F1 的置换重要性方法进行估计。结果显示，absence_above_7、absence_parent_risk、VisITedResources、Discussion、relation_mum 和 AnnouncementsView 等特征位居前列，说明缺勤情况、家长参与和学习活动行为是影响分类结果的关键因素。

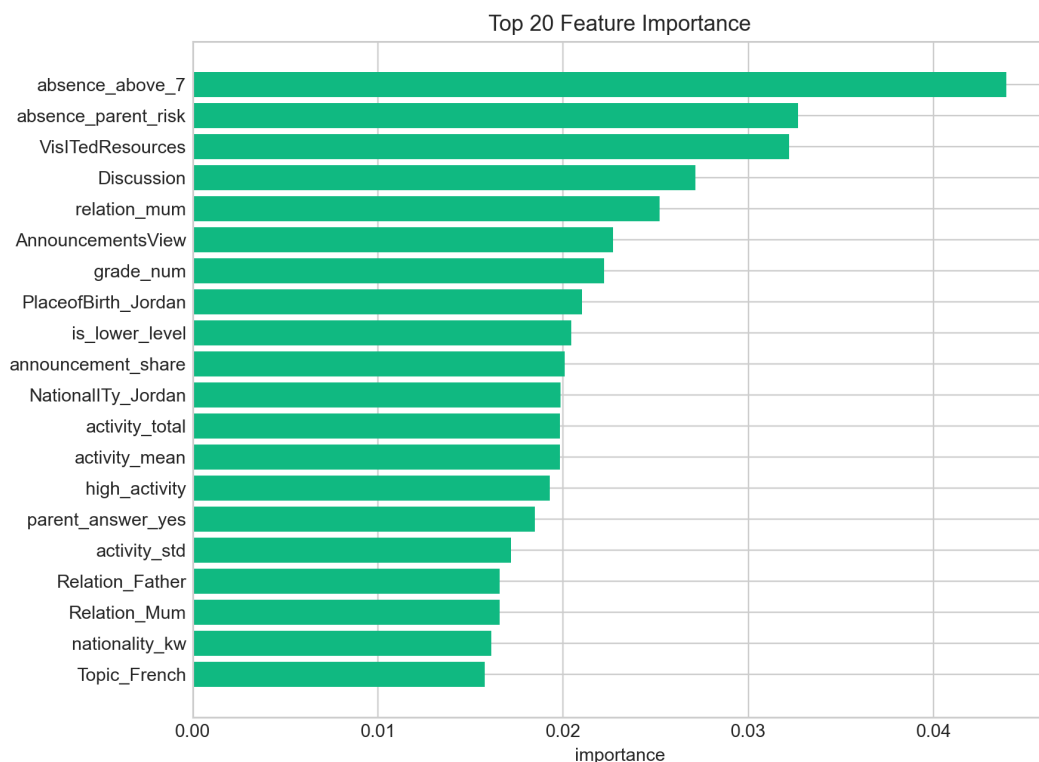


图 7: 最佳模型前 20 个重要特征

3.2 参数调优及可扩展内容

本次实验采用固定参数完成模型比较。支持向量机使用 RBF 核函数，正则化参数 $C=6.0$ ，并设置 `class_weight=balanced` 以缓解类别分布不均衡带来的影响。随机森林设置 500 棵树、最小叶节点样本数为 2，并使用类别平衡的子采样权重。后续若继续优化，可以从以下方向扩展：

- 使用网格搜索或随机搜索进一步调节 SVM 的 C 和 γ ，寻找更合适的非线性分类边界。
- 对随机森林搜索 `max_depth`、`min_samples_leaf` 和 `max_features`，在拟合能力和泛化能力之间取得平衡。
- 使用更稳定的嵌套交叉验证估计最终泛化性能，减少单次训练测试划分带来的偶然性。
- 将 `Class` 看作有序等级，尝试有序分类或先回归后分箱的方法，利用 L、M、H 之间天然存在的顺序信息。
- 加入更细粒度的时间序列学习行为，例如最近一周资源访问变化、连续缺勤次数和作业提交趋势。

4 讨论与结论

4.1 结果讨论

通过实验可以得到以下结论。第一，学习行为特征与学生成绩等级关系明显。高等级学生在举手、资源访问、公告查看和讨论参与方面均显著高于低等级学生，其中 H 类学生平均资源访问次数为 78.75，而 L 类学生仅为 18.32。这说明在线学习平台记录的行为数据能够较好反映学生学习投入程度。

第二，缺勤和家校互动信息具有较高预测价值。置换重要性结果显示，`absence_above_7` 和 `absence_parent_risk` 位居前两位，说明缺勤超过 7 天会显著影响成绩等级判断；当缺勤较多且家长问卷未参与时，学生更可能处于风险状态。这一结果也符合教育场景中的常识。

第三，非线性模型整体优于线性模型。`SVM_RBF` 的测试集 Macro-F1 达到 0.8090，高于逻辑回归的 0.7385；随机森林和梯度提升也优于逻辑回归。这说明学生成绩等级并不是由单一特征线性决定，而是由学习行为、缺勤、年级、科目和家庭支持等因素共同影响。

第四，中间等级 M 最容易发生混淆。混淆矩阵显示，M 类有一部分被误判为 L 或 H，这是因为中等表现学生本来就处于低高等级之间，行为特征可能同时具备两侧特征。实际应用时，可以将这类学生作为重点观察对象，而不只看模型给出的单一类别。

4.2 结论总结

本次实验完成了基于 Kalboard 360 学习管理系统数据的学生成绩等级分类任务。实验从 CSV 数据读取开始，依次完成数据预览、探索性分析、特征工程、类别编码、模型训练、交叉验证和结果可视化。结果表明，支持向量机 RBF 模型取得最佳测试集表现，准确率为 0.8021，Macro-F1 为 0.8090，平衡准确率为 0.8117；5 折交叉验证 Macro-F1 为 0.7902，说明模型具有一定泛化能力。

综合来看，学生成绩等级与学习活跃度、缺勤情况和家校互动密切相关。访问课程资源、参与讨论、查看公告和上课举手等行为越活跃，学生越可能处于较高成绩等级；缺勤较多且家长参与不足时，学生更容易出现低成绩风险。机器学习模型可以为教学预警提供辅助，但最终仍需要教师结合学生实际情况进行解释和干预。

5 心得体会与思考

5.1 个人心得

通过本次实验，我进一步理解了分类任务与前几次回归任务的差异。回归任务更关注数值误差大小，而分类任务除了整体准确率，还要关注每个类别是否都能被较好

识别。对于本数据集，如果只看准确率，可能会忽略中间等级和低等级学生的识别情况；使用 Macro-F1 和混淆矩阵后，模型的优点和不足会更清楚。

本次实验也让我体会到，特征工程不只是机械地增加变量，而是要结合数据含义进行设计。例如，`absence_parent_risk` 将缺勤和家长问卷信息结合起来，反映了学生学习风险和家庭支持不足的叠加效应。这个特征在置换重要性中排名靠前，说明有业务含义的组合特征确实可能帮助模型。

5.2 启发与思考

本实验中的模型可以作为教学预警的初步工具，但不能直接替代教师判断。学生成绩受到很多因素影响，包括学习基础、家庭环境、课堂氛围、考试难度和心理状态等，而数据集中只包含学习平台记录的一部分信息。因此，模型预测为低等级并不意味着对学生能力的定性评价，而应被理解为“需要更多关注”的信号。

后续如果继续改进，我希望尝试两类方向：一是收集时间序列数据，观察学生学习行为随时间的变化，而不仅仅使用静态汇总特征；二是将模型输出从单一类别扩展为风险概率，让教师看到模型的不确定性。例如，当某名学生处于 M 和 L 的边界附近时，系统可以提示教师进一步查看其缺勤和资源访问情况，从而做出更细致的教学干预。

6 附录

算法代码及其注释

```
# -*- coding: utf-8 -*-
"""
Report 4: Student performance level classification with xAPI-Edu-Data.

The script reads kalboard360_xAPI_Edu_Data.csv, performs exploratory
↪ analysis,
builds several classification models, and saves the tables/figures used by
↪ the
LaTeX report.
"""

from __future__ import annotations

import argparse
import json
import warnings
from pathlib import Path
```

```
import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import GradientBoostingClassifier,
↳ RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    accuracy_score,
    balanced_accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
)
from sklearn.model_selection import StratifiedKFold, cross_validate,
↳ train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.svm import SVC

warnings.filterwarnings("ignore")

TARGET_COL = "Class"
CLASS_ORDER = ["L", "M", "H"]
RANDOM_STATE = 42

def ensure_dir(path: Path) -> Path:
    path.mkdir(parents=True, exist_ok=True)
    return path
```

```
def make_one_hot_encoder() -> OneHotEncoder:
    try:
        return OneHotEncoder(handle_unknown="ignore", sparse_output=False)
    except TypeError:
        return OneHotEncoder(handle_unknown="ignore", sparse=False)

def load_data(data_path: Path) -> pd.DataFrame:
    if not data_path.exists():
        raise FileNotFoundError(f"Cannot find data file: {data_path}")
    df = pd.read_csv(data_path)
    expected_cols = {
        "gender",
        "NationalITy",
        "PlaceofBirth",
        "StageID",
        "GradeID",
        "SectionID",
        "Topic",
        "Semester",
        "Relation",
        "raisedhands",
        "VisITedResources",
        "AnnouncementsView",
        "Discussion",
        "ParentAnsweringSurvey",
        "ParentschoolSatisfaction",
        "StudentAbsenceDays",
        "Class",
    }
    missing_cols = expected_cols - set(df.columns)
    if missing_cols:
        raise ValueError(f"Dataset is missing columns:
        ↪ {sorted(missing_cols)}")
    return df

def add_features(df: pd.DataFrame) -> pd.DataFrame:
    data = df.copy()
```

```

activity_cols = [
    "raisedhands",
    "VisITedResources",
    "AnnouncementsView",
    "Discussion",
]
data["activity_total"] = data[activity_cols].sum(axis=1)
data["activity_mean"] = data[activity_cols].mean(axis=1)
data["activity_std"] = data[activity_cols].std(axis=1)
data["resource_per_raise"] = data["VisITedResources"] / (
    data["raisedhands"] + 1.0
)
data["discussion_per_resource"] = data["Discussion"] / (
    data["VisITedResources"] + 1.0
)
data["announcement_share"] = data["AnnouncementsView"] / (
    data["activity_total"] + 1.0
)

data["parent_answer_yes"] = (
    data["ParentAnsweringSurvey"].str.lower().eq("yes").astype(int)
)
data["school_satisfaction_good"] = (
    data["ParentschoolSatisfaction"].str.lower().eq("good").astype(int)
)
data["absence_above_7"] = (
    data["StudentAbsenceDays"].str.lower().eq("above-7").astype(int)
)
data["relation_mum"] = data["Relation"].str.lower().eq("mum").astype(int)
data["is_female"] = data["gender"].str.upper().eq("F").astype(int)
data["born_in_kuwait"] =
    ↪ data["PlaceofBirth"].str.lower().eq("kuwait").astype(int)
data["nationality_kw"] =
    ↪ data["NationalITy"].str.upper().eq("KW").astype(int)
data["grade_num"] = (
    data["GradeID"].astype(str).str.extract(r"G-(\d+)")[0].astype(float)
)
data["is_lower_level"] =
    ↪ data["StageID"].str.lower().eq("lowerlevel").astype(int)

data["high_activity"] = (

```



```

        data["activity_total"] >= data["activity_total"].median()
    ).astype(int)
    data["absence_parent_risk"] = (
        data["absence_above_7"] * (1 - data["parent_answer_yes"])
    )
    data["support_good_no_absence"] = (
        data["school_satisfaction_good"] * (1 - data["absence_above_7"])
    )
    return data

def build_preprocessor(X: pd.DataFrame) -> ColumnTransformer:
    categorical_features =
    ↪ X.select_dtypes(include=["object"]).columns.tolist()
    numeric_features = [col for col in X.columns if col not in
    ↪ categorical_features]

    numeric_pipeline = Pipeline(
        steps=[
            ("scaler", StandardScaler()),
        ]
    )
    categorical_pipeline = Pipeline(
        steps=[
            ("onehot", make_one_hot_encoder()),
        ]
    )
    return ColumnTransformer(
        transformers=[
            ("num", numeric_pipeline, numeric_features),
            ("cat", categorical_pipeline, categorical_features),
        ],
        remainder="drop",
    )

def get_feature_names(preprocessor: ColumnTransformer) -> list[str]:
    names: list[str] = []
    for transformer_name, transformer, columns in preprocessor.transformers_:
        if transformer_name == "remainder" or transformer == "drop":
            continue

```

```

        if transformer_name == "cat":
            encoder = transformer.named_steps["onehot"]
            names.extend(encoder.get_feature_names_out(columns).tolist())
        else:
            names.extend(list(columns))
    return names

def classification_metrics(y_true: np.ndarray, y_pred: np.ndarray) ->
    dict[str, float]:
    ↪ dict[str, float]:
    return {
        "Accuracy": accuracy_score(y_true, y_pred),
        "Macro_Precision": precision_score(
            y_true, y_pred, labels=CLASS_ORDER, average="macro",
            ↪ zero_division=0
        ),
        "Macro_Recall": recall_score(
            y_true, y_pred, labels=CLASS_ORDER, average="macro",
            ↪ zero_division=0
        ),
        "Macro_F1": f1_score(
            y_true, y_pred, labels=CLASS_ORDER, average="macro",
            ↪ zero_division=0
        ),
        "Balanced_Accuracy": balanced_accuracy_score(y_true, y_pred),
    }

def save_overview(df: pd.DataFrame, out_dir: Path) -> None:
    ensure_dir(out_dir)
    df.head(10).to_csv(out_dir / "head_10.csv", index=False,
    ↪ encoding="utf-8-sig")
    df.dtypes.astype(str).rename("dtype").to_csv(
        out_dir / "dtypes.csv", encoding="utf-8-sig"
    )
    df.isna().sum().rename("missing_count").to_csv(
        out_dir / "missing_values.csv", encoding="utf-8-sig"
    )
    df.describe(include="all").to_csv(out_dir / "describe.csv",
    ↪ encoding="utf-8-sig")

```

```

class_counts = df[TARGET_COL].value_counts().reindex(CLASS_ORDER)
class_counts.rename("count").to_csv(
    out_dir / "class_distribution.csv", encoding="utf-8-sig"
)

numeric_cols = [
    "raisedhands",
    "VisITedResources",
    "AnnouncementsView",
    "Discussion",
]
activity_by_class = df.groupby(TARGET_COL)[numeric_cols].agg(["mean",
↪ "median"])
activity_by_class = activity_by_class.reindex(CLASS_ORDER)
activity_by_class.to_csv(out_dir / "activity_by_class.csv",
↪ encoding="utf-8-sig")

categorical_cols = [
    "gender",
    "StageID",
    "Semester",
    "Relation",
    "ParentAnsweringSurvey",
    "ParentschoolSatisfaction",
    "StudentAbsenceDays",
]
with (out_dir / "category_crosstab.txt").open("w", encoding="utf-8") as
↪ f:
    for col in categorical_cols:
        f.write(f"\n[{col}]\n")
        f.write(pd.crosstab(df[col], df[TARGET_COL]).to_string())
        f.write("\n")

def plot_eda(df: pd.DataFrame, fig_dir: Path) -> None:
    ensure_dir(fig_dir)
    plt.style.use("seaborn-v0_8-whitegrid")

    class_counts = df[TARGET_COL].value_counts().reindex(CLASS_ORDER)
    fig, ax = plt.subplots(figsize=(5.8, 4.0))

```

```

ax.bar(class_counts.index, class_counts.values, color=["#EF4444",
↪ "#3B82F6", "#10B981"])
ax.set_title("Class Distribution")
ax.set_xlabel("performance class")
ax.set_ylabel("count")
for i, value in enumerate(class_counts.values):
    ax.text(i, value + 3, str(int(value)), ha="center")
fig.tight_layout()
fig.savefig(fig_dir / "class_distribution.png", dpi=220)
plt.close(fig)

numeric_cols = [
    "raisedhands",
    "VisITedResources",
    "AnnouncementsView",
    "Discussion",
]
means = df.groupby(TARGET_COL)[numeric_cols].mean().reindex(CLASS_ORDER)
fig, ax = plt.subplots(figsize=(8.0, 4.6))
x = np.arange(len(numeric_cols))
width = 0.24
colors = ["#EF4444", "#3B82F6", "#10B981"]
for offset, class_name, color in zip([-width, 0, width], CLASS_ORDER,
↪ colors):
    ax.bar(x + offset, means.loc[class_name].values, width,
↪ label=class_name, color=color)
ax.set_xticks(x)
ax.set_xticklabels(numeric_cols, rotation=15, ha="right")
ax.set_title("Learning Activity Mean by Class")
ax.set_ylabel("mean value")
ax.legend(title="Class")
fig.tight_layout()
fig.savefig(fig_dir / "activity_mean_by_class.png", dpi=220)
plt.close(fig)

fig, axes = plt.subplots(2, 2, figsize=(8.6, 6.2))
axes = axes.ravel()
for ax, col in zip(axes, numeric_cols):
    data = [df.loc[df[TARGET_COL] == cls, col] for cls in CLASS_ORDER]
    ax.boxplot(data, labels=CLASS_ORDER, patch_artist=True)
    ax.set_title(col)

```

```

        ax.set_xlabel("Class")
        ax.set_ylabel("value")
    fig.tight_layout()
    fig.savefig(fig_dir / "activity_boxplots_by_class.png", dpi=220)
    plt.close(fig)

    numeric_all = df.select_dtypes(include=[np.number]).columns.tolist()
    corr_df = df[numeric_all + [TARGET_COL]].copy()
    corr_df["class_code"] = corr_df[TARGET_COL].map({"L": 0, "M": 1, "H": 2})
    corr = corr_df[numeric_all + ["class_code"]].corr()
    corr.to_csv(fig_dir.parent / "results_student_performance" /
        ↪ "01_data_overview" / "numeric_correlation.csv",
        ↪ encoding="utf-8-sig")

    fig, ax = plt.subplots(figsize=(8.2, 6.4))
    im = ax.imshow(corr, cmap="RdBu_r", vmin=-1, vmax=1)
    ax.set_xticks(np.arange(len(corr.columns)))
    ax.set_yticks(np.arange(len(corr.index)))
    ax.set_xticklabels(corr.columns, rotation=60, ha="right", fontsize=8)
    ax.set_yticklabels(corr.index, fontsize=8)
    ax.set_title("Numeric Feature Correlation")
    fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)
    fig.tight_layout()
    fig.savefig(fig_dir / "correlation_heatmap.png", dpi=220)
    plt.close(fig)

def build_models(preprocessor: ColumnTransformer) -> dict[str, Pipeline]:
    return {
        "Majority_Baseline": Pipeline(
            steps=[
                ("preprocess", preprocessor),
                ("model", DummyClassifier(strategy="most_frequent")),
            ],
        ),
        "LogisticRegression": Pipeline(
            steps=[
                ("preprocess", preprocessor),
                (
                    "model",
                    LogisticRegression(

```

```

        C=1.8,
        max_iter=3000,
        class_weight="balanced",
        multi_class="auto",
        random_state=RANDOM_STATE,
    ),
),
]
),
"SVM_RBF": Pipeline(
    steps=[
        ("preprocess", preprocessor),
        (
            "model",
            SVC(
                C=6.0,
                gamma="scale",
                kernel="rbf",
                class_weight="balanced",
                probability=True,
                random_state=RANDOM_STATE,
            ),
        ),
    ],
),
"RandomForest": Pipeline(
    steps=[
        ("preprocess", preprocessor),
        (
            "model",
            RandomForestClassifier(
                n_estimators=500,
                max_depth=None,
                min_samples_leaf=2,
                class_weight="balanced_subsample",
                random_state=RANDOM_STATE,
                n_jobs=-1,
            ),
        ),
    ],
),

```

```

        "GradientBoosting": Pipeline(
            steps=[
                ("preprocess", preprocessor),
                (
                    "model",
                    GradientBoostingClassifier(
                        n_estimators=160,
                        learning_rate=0.05,
                        max_depth=3,
                        random_state=RANDOM_STATE,
                    ),
                ),
            ],
        ),
    }

def evaluate_models(
    models: dict[str, Pipeline],
    X_train: pd.DataFrame,
    X_test: pd.DataFrame,
    y_train: pd.Series,
    y_test: pd.Series,
    out_dir: Path,
    fig_dir: Path,
) -> tuple[str, Pipeline, pd.DataFrame, pd.DataFrame]:
    ensure_dir(out_dir)
    ensure_dir(fig_dir)
    rows: list[dict[str, float | str]] = []
    cv_rows: list[dict[str, float | str]] = []
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
    scoring = {
        "accuracy": "accuracy",
        "macro_f1": "f1_macro",
        "balanced_accuracy": "balanced_accuracy",
    }

    best_name = ""
    best_model: Pipeline | None = None
    best_macro_f1 = -np.inf

```

```

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    metrics = classification_metrics(y_test, y_pred)
    rows.append({"Model": name, **metrics})

cv_result = cross_validate(
    model,
    pd.concat([X_train, X_test], axis=0),
    pd.concat([y_train, y_test], axis=0),
    cv=cv,
    scoring=scoring,
    n_jobs=None,
)
cv_rows.append(
    {
        "Model": name,
        "CV_Accuracy_Mean": float(cv_result["test_accuracy"].mean()),
        "CV_Macro_F1_Mean": float(cv_result["test_macro_f1"].mean()),
        "CV_Balanced_Accuracy_Mean": float(
            cv_result["test_balanced_accuracy"].mean()
        ),
    }
)

report = classification_report(
    y_test,
    y_pred,
    labels=CLASS_ORDER,
    target_names=CLASS_ORDER,
    digits=4,
    zero_division=0,
)
(out_dir / f"{name}_classification_report.txt").write_text(
    report, encoding="utf-8"
)

if metrics["Macro_F1"] > best_macro_f1:
    best_macro_f1 = metrics["Macro_F1"]
    best_name = name
    best_model = model

```



```

results = pd.DataFrame(rows).sort_values(
    ["Macro_F1", "Accuracy"], ascending=False
)
cv_results = pd.DataFrame(cv_rows).sort_values(
    ["CV_Macro_F1_Mean", "CV_Accuracy_Mean"], ascending=False
)
results.to_csv(out_dir / "model_comparison.csv", index=False,
    ↪ encoding="utf-8-sig")
cv_results.to_csv(
    out_dir / "cross_validation_results.csv", index=False,
    ↪ encoding="utf-8-sig"
)

assert best_model is not None
y_best = best_model.predict(X_test)
pd.DataFrame({"y_true": y_test.values, "y_pred": y_best}).to_csv(
    out_dir / "best_model_predictions.csv", index=False,
    ↪ encoding="utf-8-sig"
)

cm = confusion_matrix(y_test, y_best, labels=CLASS_ORDER)
cm_df = pd.DataFrame(cm, index=CLASS_ORDER, columns=CLASS_ORDER)
cm_df.to_csv(out_dir / "best_confusion_matrix.csv", encoding="utf-8-sig")

fig, ax = plt.subplots(figsize=(5.5, 4.8))
ConfusionMatrixDisplay(confusion_matrix=cm,
    ↪ display_labels=CLASS_ORDER).plot(
    ax=ax, cmap="Blues", colorbar=False, values_format="d"
)
ax.set_title(f"Confusion Matrix: {best_name}")
fig.tight_layout()
fig.savefig(fig_dir / "best_confusion_matrix.png", dpi=220)
plt.close(fig)

fig, ax = plt.subplots(figsize=(7.0, 4.2))
plot_results = results.sort_values("Macro_F1", ascending=True)
ax.barh(plot_results["Model"], plot_results["Macro_F1"], color="#2563EB")
ax.set_xlim(0, 1.0)
ax.set_xlabel("Macro-F1")
ax.set_title("Model Comparison")

```

```

    for i, value in enumerate(plot_results["Macro_F1"]):
        ax.text(value + 0.01, i, f"{value:.3f}", va="center")
    fig.tight_layout()
    fig.savefig(fig_dir / "model_comparison_macro_f1.png", dpi=220)
    plt.close(fig)

    return best_name, best_model, results, cv_results

def save_feature_importance(
    best_model: Pipeline,
    X_test: pd.DataFrame,
    y_test: pd.Series,
    out_dir: Path,
    fig_dir: Path,
) -> pd.DataFrame:
    preprocess = best_model.named_steps["preprocess"]
    model = best_model.named_steps["model"]
    feature_names = get_feature_names(preprocess)

    if hasattr(model, "feature_importances_"):
        values = model.feature_importances_
        source = "model_importance"
    else:
        transformed = preprocess.transform(X_test)
        perm = permutation_importance(
            model,
            transformed,
            y_test,
            scoring="f1_macro",
            n_repeats=20,
            random_state=RANDOM_STATE,
        )
        values = perm.importances_mean
        source = "permutation_importance"

    importance = (
        pd.DataFrame({"feature": feature_names, "importance": values,
            ↵ "source": source})
        .sort_values("importance", ascending=False)
        .reset_index(drop=True)

```

```

    )
    importance.to_csv(out_dir / "feature_importance_top20.csv", index=False,
        ↪ encoding="utf-8-sig")

    top = importance.head(20).iloc[::-1]
    fig, ax = plt.subplots(figsize=(8.2, 6.0))
    ax.barh(top["feature"], top["importance"], color="#10B981")
    ax.set_title("Top 20 Feature Importance")
    ax.set_xlabel("importance")
    fig.tight_layout()
    fig.savefig(fig_dir / "feature_importance_top20.png", dpi=220)
    plt.close(fig)
    return importance

def run(data_path: Path, output_root: Path) -> None:
    result_dir = ensure_dir(output_root / "results_student_performance")
    overview_dir = ensure_dir(result_dir / "01_data_overview")
    model_dir = ensure_dir(result_dir / "02_modeling")
    fig_dir = ensure_dir(output_root / "figures")

    df = load_data(data_path)
    engineered = add_features(df)
    save_overview(df, overview_dir)
    plot_eda(df, fig_dir)

    X = engineered.drop(columns=[TARGET_COL])
    y = engineered[TARGET_COL]
    X_train, X_test, y_train, y_test = train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=RANDOM_STATE,
        stratify=y,
    )

    preprocessor = build_preprocessor(X)
    models = build_models(preprocessor)
    best_name, best_model, results, cv_results = evaluate_models(
        models, X_train, X_test, y_train, y_test, model_dir, fig_dir
    )

```

```

importance = save_feature_importance(best_model, X_test, y_test,
    ↪ model_dir, fig_dir)

summary = {
    "data_shape": list(df.shape),
    "train_size": int(len(X_train)),
    "test_size": int(len(X_test)),
    "class_distribution":
    ↪ df[TARGET_COL].value_counts().reindex(CLASS_ORDER).to_dict(),
    "best_model": best_name,
    "best_test_metrics": results.loc[results["Model"] ==
    ↪ best_name].iloc[0].to_dict(),
    "best_cv_metrics": cv_results.loc[cv_results["Model"] ==
    ↪ best_name].iloc[0].to_dict(),
    "top_features": importance.head(10)[["feature",
    ↪ "importance"]].to_dict("records"),
}

(model_dir / "summary.json").write_text(
    json.dumps(summary, ensure_ascii=False, indent=2), encoding="utf-8"
)

print("Data shape:", df.shape)
print("Class distribution:")
print(df[TARGET_COL].value_counts().reindex(CLASS_ORDER).to_string())
print("\nModel comparison:")
print(results.to_string(index=False, float_format=lambda x: f"{x:.4f}"))
print("\nCross validation:")
print(cv_results.to_string(index=False, float_format=lambda x:
    ↪ f"{x:.4f}"))
print("\nBest model:", best_name)
print("Top features:")
print(importance.head(10).to_string(index=False, float_format=lambda x:
    ↪ f"{x:.4f}"))

def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser()
    parser.add_argument(
        "--data",
        type=Path,
        default=Path("kalboard360_xAPI_Edu_Data.csv"),

```

```
        help="Path to xAPI Edu Data CSV file.",
    )
    parser.add_argument(
        "--output",
        type=Path,
        default=Path("."),
        help="Directory for generated report tables and figures.",
    )
    return parser.parse_args()

if __name__ == "__main__":
    args = parse_args()
    run(args.data, args.output)
```

关键运行结果汇总

数据规模: (480, 17)

目标变量: Class, 类别为 L/M/H

训练集样本数: 384

测试集样本数: 96

类别分布:

L: 127

M: 211

H: 142

测试集模型结果:

SVM_RBF Accuracy=0.8021, Macro-F1=0.8090, Balanced

↪ Accuracy=0.8117

RandomForest Accuracy=0.7604, Macro-F1=0.7707, Balanced

↪ Accuracy=0.7764

GradientBoosting Accuracy=0.7396, Macro-F1=0.7513, Balanced

↪ Accuracy=0.7588

LogisticRegression Accuracy=0.7292, Macro-F1=0.7385, Balanced

↪ Accuracy=0.7580

Majority_Baseline Accuracy=0.4375, Macro-F1=0.2029, Balanced

↪ Accuracy=0.3333

SVM_RBF 测试集分类报告:

L: precision=0.8462, recall=0.8800, f1-score=0.8627, support=25

M: precision=0.7805, recall=0.7619, f1-score=0.7711, support=42

H: precision=0.7931, recall=0.7931, f1-score=0.7931, support=29

最佳模型混淆矩阵:

	预测 L	预测 M	预测 H
真实 L	22	3	0
真实 M	4	32	6
真实 H	0	6	23

重要特征前五:

absence_above_7=0.0439

absence_parent_risk=0.0327

VisITedResources=0.0322

Discussion=0.0272

relation_mum=0.0252